Miniproject 1 (Warmup)
Life expectancy and schooling

September 2026

Introduction

The World Health Organization (WHO) provide public data that disaggregate health metrics, such as life expectancy, child mortality, disease burden, alcohol use, by life expectancy, economic status etc. The data covers 193 countries. In this assignment you will work with this data and specifically study the relationship between life expectancy and schooling. It is of course possible to analyse many different relationships in the data, but it is not part of this assignment.
The aim here is to compare algorithms that are at play under the hood, when working with regression analysis. You can use the built-in methods in `numpy` and/or `scipy` for solving equation systems, QR-factorization etc.

To Do

1. Download the data file `LifeExpectancyData.csv` from the course page in Studium. The file contains public data from WHO as described in the introduction above.

   (a) **Clean up the data**
   We will only analyse the dependency between *life expectancy* and *Schooling*, i.e. only two columns. Start your Python program by importing these two columns from the data.
   There are also some NaN-values in the data (=missing data). You can deal with missing data in different ways, but the easiest is to simply remove these lines in the data (note, you must remove in both columns). You can remove NaN-values with the code `cleaned_data = data[~np.isnan(data).any(axis=1)]`.
   Finally, it is a good idea to plot the data, to see if it looks reasonable.

   (b) Analyse the dependency between life expectancy and schooling. Use quadratic regression, where schooling is the independent variable (x) and life expectancy the dependent variable (y). Solve the least squares problem in two different ways, by forming and solving the normal equations ($A^TAx = A^Ty$) and also via QR-decomposition. Plot the polynomials together with the data set.

(c) In ordinary least squares regression, analysts typically assume the residual follows a normal distribution centered at the mean of zero (this is called the *n*ormality assumption). Is that the case here? Create a histogram of the residual and examine visually if it roughly looks like a normal distribution. Don't use too few bins in the histogram, e.g. 200 might be good.

(d) Calculate the condition number that is relevant to this problem. How much accuracy would you possibly loose (in the worst case) in the normal equation solution x_{NE}?

How much accuracy did we actually loose? We don't have the exact solution available, but the QR-based solution should be better than the one based on the Normal equations. One idea is to consider the QR-solution as "exact" (it isn't of course but it is closer to the exact solution). Use this idea and estimate the absolute and the relative error in x_{NE} (solution based on the Normal equations). Compare with the interpretation of the condition number. (Remember, no error at all is equivalent to relative error $\approx 10^{-16}$.)

2. Improve your code in (1), and make possible to change polynomial degree just by changing one variable, e.g. `pol_degree`. For example, `pol_degree=1` would run the steps in 1(a) to 1(d) using a first order polynomial.
Try first order and also 3rd order polynomials and compare the results you get (c) and (d). Does the condition number get smaller/larger? How does the normal distribution change?